

Apostila Java — Fundamentos

Guia de referência rápida com as principais sintaxes estudadas, dicas profissionais e insights. Baseado nos exercícios práticos do repositório `Pasta_Java_Exercicios`.

Sumário

1. Estrutura básica de um programa
2. Variáveis e tipos de dados
3. Entrada de dados com Scanner
4. Saída formatada com printf
5. Locale — ponto vs vírgula
6. Operadores
7. Estruturas condicionais
8. Estruturas de repetição
9. Strings — principais métodos
10. Classe Character
11. Arrays (vetores)
12. Matrizes (arrays 2D)
13. Tratamento de exceções
14. TimeUnit e pausas no programa
15. Datas em Java
16. Dicas profissionais e insights

1. Estrutura básica de um programa

```
package exercicios.basicos;    // pacote (pasta lógica) da classe

import java.util.Scanner;      // imports vêm depois do package

public class MinhaClasse {     // nome da classe = nome do arquivo
    public static void main(String[] args) { // ponto de entrada do programa
        System.out.println("Olá, Java!");
    }
}
```

Regras importantes: - O nome do arquivo deve ser **idêntico** ao nome da classe pública (`MinhaClasse.java`) - Nomes de classe usam **PascalCase** (`Ex10EscalandoOperacoes`) - Nomes de variáveis e métodos usam **camelCase** (`temLetra` , `escolhaTemp`) - Pacotes usam **minúsculas** (`exercicios.basicos`)

⚠ **Nunca** nomeie um pacote começando com `java.` — é um namespace reservado pela JVM e gera `SecurityException: Prohibited package name` em tempo de execução.

2. Variáveis e tipos de dados

Tipos primitivos mais usados

Tipo	Descrição	Exemplo
<code>int</code>	inteiro (32 bits)	<code>int idade = 30;</code>
<code>long</code>	inteiro grande (64 bits)	<code>long populacao = 8_000_000_000L;</code>
<code>double</code>	decimal (64 bits)	<code>double altura = 1.75;</code>
<code>float</code>	decimal (32 bits)	<code>float peso = 70.5f;</code>
<code>char</code>	um único caractere	<code>char letra = 'A';</code>
<code>boolean</code>	verdadeiro/falso	<code>boolean ativo = true;</code>

String (tipo referência, não primitivo)

```
String nome = "Jader";
```

Declaração múltipla na mesma linha

```
int a = 10, b = 3;    // válido quando as variáveis são relacionadas
```

💡 **Insight:** use o tipo que representa a semântica do dado. Um **contador** de loop deve ser `int`, não `double` — mesmo que `double` "funcione". Código profissional comunica intenção.

3. Entrada de dados com Scanner

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

int inteiro    = sc.nextInt();
double real    = sc.nextDouble();
String linha   = sc.nextLine(); // lê a linha inteira, incluindo espaços
String token   = sc.next();      // lê só até o próximo espaço

sc.close(); // sempre feche o Scanner ao final
```

⚠ Armadilha clássica: buffer do Scanner

Quando `nextInt()` / `nextDouble()` é seguido de `nextLine()`, a quebra de linha "sobra" no buffer e o `nextLine()` captura uma string vazia. Solução:

```
int idade = sc.nextInt();
sc.nextLine(); // limpa o buffer
String nome = sc.nextLine(); // agora lê corretamente
```

Normalização de entrada (padrão recomendado)

```
String entrada = sc.nextLine().trim().toUpperCase();
// trim() remove espaços das bordas; toUpperCase() padroniza a comparação
```

4. Saída formatada com printf

Especificadores de formato

Especificador	Tipo	Exemplo de saída
%d	inteiro	42
%f	decimal (6 casas por padrão)	3.140000
%.2f	decimal com 2 casas	3.14
%s	string	Java
%c	caractere	A
%b	booleano	true
%n	quebra de linha (portável)	—
%%	símbolo % literal	%

Modificadores de largura e alinhamento

```
System.out.printf("%5d", 7);    // "   7" (largura 5, alinhado à direita)
System.out.printf("%-5d", 7);   // "7   " (alinhado à esquerda)
System.out.printf("%03d", 7);   // "007" (zeros à esquerda)
System.out.printf("%-8.2f", x); // decimal 2 casas, coluna de largura 8
```

Exemplo completo

```
System.out.printf("%.2f x %d = %-8.2f%n", num, i, resultado);
```

⚠ %d com double (ou %f com int) lança `IllegalFormatConversionException`. O especificador precisa bater com o tipo passado.

💡 Prefira %n a \n no printf — ele se adapta ao sistema operacional (Windows usa \r\n, Linux/Mac usam \n).

5. Locale — ponto vs vírgula

O Brasil usa vírgula como separador decimal; o padrão internacional (e de programação) usa ponto. Sem configurar, `Scanner` e `printf` seguem o locale do sistema (`pt_BR`).

Solução recomendada: definir o Locale da JVM inteira

```
import java.util.Locale;

public static void main(String[] args) {
    Locale.setDefault(Locale.US); // PRIMEIRA linha do main
    Scanner sc = new Scanner(System.in);
    // ...
}
```

⚠ **A ordem importa:** o `Scanner` captura o Locale **no momento da criação**. Se `Locale.setDefault()` vier depois do `new Scanner(...)`, a entrada continuará esperando vírgula. Já o `printf` consulta o Locale a cada chamada.

6. Operadores

Aritméticos

```
+ - * / %          // % é o resto da divisão (módulo)

10 / 3    // 3    (divisão inteira quando ambos são int!)
10.0 / 3  // 3.333... (basta um ser double)
10 % 3    // 1    (resto)
```

⚠ **Divisão inteira** é fonte clássica de bug: `(double) soma / cont` — o cast em um dos operandos força a divisão decimal.

Incremento e decremento

```
i++ // pós-incremento: usa o valor, DEPOIS incrementa
++i // pré-incremento: incrementa, DEPOIS usa o valor
i-- // pós-decremento
--i // pré-decremento

int a = 5;
System.out.println(a++); // imprime 5, a vira 6
System.out.println(++a); // a vira 7, imprime 7
```

Atribuição composta

```
x += 5; // x = x + 5
x -= 5; // x = x - 5
x *= 2; // x = x * 2 → útil p/ crescimento percentual: pop *= 1.015
x /= 2; // x = x / 2
```

Relacionais

```
== != > < >= <=
```

⚠ Para **Strings**, use `.equals()`, nunca `==`: `java senha.equals("99")` // ✓ compara conteúdo `senha == "99"` // ✗ compara referência de memória

Lógicos

Operador	Nome	Comportamento
&&	E (curto-circuito)	se o 1º for <code>false</code> , nem avalia o 2º
\ \	OU (curto-circuito)	se o 1º for <code>true</code> , nem avalia o 2º
!	NÃO	inverte o booleano
^	XOR	<code>true</code> somente se os dois forem diferentes
& / \	E/OU sem curto-circuito	avaliam sempre os dois lados

🧠 Lei de De Morgan (validação de entrada)

Para aceitar **somente** 1 ou 2:

```
// ✖ ERRADO – sempre true, loop infinito:
while (opcao != 1 || opcao != 2)

// ✔ CORRETO – "não é 1 E não é 2":
while (opcao != 1 && opcao != 2)
```

Regra: `NÃO(A OU B)` equivale a `(NÃO A) E (NÃO B)`.

Curto-circuito como proteção

```
// length() != 1 protege o charAt(0) de StringIndexOutOfBoundsException:
while (letra.length() != 1 || !Character.isLetter(letra.charAt(0)))
```

7. Estruturas condicionais

if / else if / else

```
if (nota >= 7) {
    System.out.println("Aprovado");
} else if (nota >= 5) {
    System.out.println("Recuperação");
} else {
    System.out.println("Reprovado");
}
```

switch

```
switch (opcao) {
    case 1:
        System.out.println("Celsius");
        break;           // sem break, "vaza" para o próximo case!
    case 2:
        System.out.println("Fahrenheit");
        break;
    default:
        System.out.println("Opção inválida.");
}
```

⚠ `switch (c = 1)` é **atribuição**, não comparação — o correto é `switch (c)`. O `switch` compara automaticamente com cada `case`.

Operador ternário

```
String resultado = (nota >= 7) ? "Aprovado" : "Reprovado";
```

💡 Ternário para **atribuições simples**; `if/else` para blocos com múltiplas instruções.

8. Estruturas de repetição

for (contador conhecido)

```
for (int i = 1; i <= 10; i++) {
    System.out.printf("%d x %d = %d\n", n, i, n * i);
}
```

⚠ **Off-by-one:** `i < 10` roda 9 vezes; `i <= 10` roda 10. Erro clássico — sempre confira o limite.

for-each (percorrer coleções/arrays sem índice)

```
for (int valor : vetor) {
    System.out.println(valor);
}
```

while (testa antes de executar)

```
while (senha.isEmpty()) {
    System.out.println("Entrada inválida. Digite novamente:");
    senha = sc.nextLine().trim();
}
```

do-while (executa pelo menos uma vez)

```
do {
    System.out.println(menu);
    opcao = sc.nextLine().trim().toUpperCase();
} while (opcao.isEmpty());
```

break e continue

```
break;    // sai do loop imediatamente
continue; // pula para a próxima iteração
```

⚠ Dentro de um `switch` que está dentro de um loop, o `break` sai só do `switch`. Para sair do loop, use uma **flag booleana** (`boolean valido = false; ... valido = true;`).

🔑 Padrão de validação com flags

```
boolean temLetra = false;
boolean temNumero = false;

for (int i = 0; i < senha.length(); i++) {
    char c = senha.charAt(i);
    if (Character.isLetter(c)) temLetra = true;
    if (Character.isDigit(c)) temNumero = true;
}

if (senha.length() >= 8 && temLetra && temNumero) {
    System.out.println("Senha válida!");
}
```

⚠ **Se o processo repetir (while externo), resete as flags para `false` a cada tentativa** — senão uma tentativa anterior "contamina" a próxima.

9. Strings — principais métodos

```
String s = "Programacao";

s.length()           // 11 – tamanho
s.charAt(0)          // 'P' – caractere na posição (índice começa em 0!)
s.toUpperCase()       // "PROGRAMACAO"
s.toLowerCase()      // "programacao"
s.trim()              // remove espaços das bordas
s.isEmpty()           // true se for ""
s.equals("abc")       // comparação de conteúdo
s.equalsIgnoreCase("ABC") // ignora maiúsc./minúsc.
s.contains("gra")     // true – contém a substring? (recebe String, não char)
s.indexOf('P')        // 0 – posição da 1ª ocorrência; -1 se não existir
s.lastIndexOf('a')    // posição da última ocorrência
s.substring(0, 4)     // "Prog" – do índice 0 até o 3 (o fim é exclusivo)
s.replace('a', '@')   // "Progr@mc@o"
s.split(" ")          // divide em array de Strings pelo separador
s.repeat(3)           // repete a string 3x (Java 11+)
s.toCharArray()      // converte em char[] para percorrer letra a letra
```

🔗 Por que `indexOf` retorna `-1`?

Índices válidos vão de `0` a `length()-1` — nunca negativos. O `-1` é um **valor sentinela** inequívoco de "não encontrado" (não pode ser `null` porque `int` é primitivo):

```
if ("AEIOU".indexOf(c) != -1) { // c foi encontrado → é vogal
```

⚠ Armadilhas

```
"AEIOU".contains("") // TRUE! Toda string contém a string vazia
"AEIOU".contains(c)  // ERRO se c for char – contains exige String
"AEIOU".indexOf(c)   // ✔ indexOf tem versão que aceita char
```

Multiplicar/repetir strings

```
String linha = "=".repeat(25); // "=====
```

⚠ Evite `+=` de String dentro de loops grandes (String é **imutável**; cada concatenação cria um objeto novo). Para montagem intensa, use `StringBuilder`: `java StringBuilder sb = new StringBuilder(); for (int i = 0; i < 1000; i++) sb.append("x"); String resultado = sb.toString();`

10. Classe Character

Métodos estáticos para analisar um `char`:

```

Character.isLetter('A')      // true – é letra? (inclui acentuadas: 'ç' → true)
Character.isDigit('5')      // true – é dígito 0-9?
Character.isLetterOrDigit('@') // false
Character.isUpperCase('A')  // true
Character.isLowerCase('a')  // true
Character.isWhitespace(' ') // true – espaço, tab, quebra de linha
Character.toUpperCase('a')   // 'A'
Character.toLowerCase('A')  // 'a'

```

Padrão de uso combinado com charAt

```

char c = texto.charAt(i);
if (Character.isLetter(c)) { ... }

```

11. Arrays (vetores)

Declaração e inicialização

```

// Com valores definidos
int[] vet = {10, 20, 30, 40, 50};

// Com tamanho definido (preenchido com 0 / 0.0 / false / null)
double[] notas = new double[5];

// △ Sintaxe correta na declaração:
double[] arr = new double[n]; // ✓
double arr[i];                // ✕ erro comum

```

Acesso e tamanho

```

vet[0]          // primeiro elemento (índice começa em 0)
vet[vet.length - 1] // último elemento
vet.length      // tamanho (SEM parênteses – é atributo, não método)

```

Preenchendo com Scanner

```

double[] vet = new double[5];
for (int i = 0; i < vet.length; i++) {
    System.out.printf("Digite o %dº valor: ", i + 1); // i+1 p/ exibição amigável
    vet[i] = sc.nextDouble();
}

```

Percorrendo

```

// Com índice (quando a posição importa)
for (int i = 0; i < vet.length; i++) {
    System.out.println("vet[" + i + "] = " + vet[i]);
}

// for-each (quando só o valor importa)
for (double v : vet) {
    System.out.println(v);
}

```


Imprimindo rápido (debug)

```
import java.util.Arrays;

System.out.println(Arrays.toString(vet)); // [10, 20, 30, 40, 50]
```

Padrão acumulador (soma / média / contagem)

```
int soma = 0;
int cont = 0;
for (int i = 0; i < vet.length; i++) {
    if (vet[i] % 2 != 0) { // só ímpares
        soma += vet[i];
        cont++;
    }
}
double media = (cont != 0) ? (double) soma / cont : 0;
```

⚠ **Escopo de variável:** declare a variável no escopo onde ela será **usada**. Redefinir `double media` dentro de um `if` cria uma variável nova local ao bloco — a de fora continua `0`.

Métodos úteis da classe Arrays

```
Arrays.toString(vet) // representação em texto
Arrays.sort(vet) // ordena crescente (modifica o array!)
Arrays.fill(vet, 0) // preenche tudo com um valor
Arrays.copyOf(vet, tamanho) // copia para um novo array
Arrays.stream(vet).sum() // soma (streams)
Arrays.stream(vet).average().orElse(0) // média como double
```

12. Matrizes (arrays 2D)

Em Java, matriz é um **array de arrays**.

Declaração

```
// Com valores
int[][] mat = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Com dimensões (preenchida com 0)
int[][] mat = new int[3][4]; // 3 linhas, 4 colunas
```

Acesso e dimensões

```
mat[1][2] // linha 1, coluna 2 → 6
mat.length // número de LINHAS → 3
mat[i].length // número de COLUNAS da linha i
```

🔗 **Por que `mat[i].length` e não `mat[j].length`?** Quando o `j` itera, você já está *dentro* da linha `i` — precisa do tamanho *daquela linha*. Isso também faz o código funcionar com **matrizes irregulares** (jagged arrays), onde cada linha pode ter tamanho diferente.

Percorrendo (for aninhado — padrão fundamental)

```
for (int i = 0; i < mat.length; i++) {           // linhas
    for (int j = 0; j < mat[i].length; j++) { // colunas da linha i
        System.out.printf("%4d", mat[i][j]); // %4d alinha as colunas
    }
    System.out.println(); // quebra de linha ao fim de cada linha da matriz
}
```

Impressão rápida (debug)

```
System.out.println(Arrays.deepToString(mat)); // [[1, 2, 3], [4, 5, 6], ...]
// deepToString para 2D; toString simples imprimiria endereços de memória
```

Preenchendo com números aleatórios

```
import java.util.Random;

Random random = new Random();
for (int i = 0; i < mat.length; i++) {
    for (int j = 0; j < mat[i].length; j++) {
        mat[i][j] = random.nextInt(100); // 0 a 99
    }
}
```

Isolando linha ou coluna

```
// Linha 1 inteira (fixa i, varia j):
for (int j = 0; j < mat[1].length; j++) {
    System.out.print(mat[1][j] + " ");
}

// Coluna 2 inteira (fixa j, varia i):
for (int i = 0; i < mat.length; i++) {
    System.out.print(mat[i][2] + " ");
}
```

🔗 **Regra de bolso:** quer uma **linha**? Fixe `i`. Quer uma **coluna**? Fixe `j`.

13. Tratamento de exceções

try / catch básico

```
try {
    int idade = sc.nextInt();
} catch (InputMismatchException e) {
    System.out.println("Digite apenas números!");
    sc.nextLine(); // limpa o buffer para não entrar em loop infinito
}
```

```
import java.util.InputMismatchException;
```

Exceções comuns no início dos estudos

Exceção	Causa típica
<code>InputMismatchException</code>	Scanner esperava um tipo, usuário digitou outro
<code>StringIndexOutOfBoundsException</code>	<code>charAt(i)</code> com índice fora da string
<code>ArrayIndexOutOfBoundsException</code>	acesso a <code>vet[i]</code> fora dos limites
<code>IllegalFormatConversionException</code>	<code>%d</code> com double, <code>%f</code> com int
<code>NullPointerException</code>	uso de referência não inicializada
<code>ArithmeticException</code>	divisão inteira por zero

💡 Antes de dividir por uma variável, valide: `if (divisor != 0) { ... }`.

14. TimeUnit e pausas no programa

Thread.sleep (milissegundos)

```
try {
    Thread.sleep(1000); // pausa 1 segundo
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // forma correta de tratar
}
```

TimeUnit (mais legível)

```
import java.util.concurrent.TimeUnit;

try {
    TimeUnit.SECONDS.sleep(1);
    TimeUnit.MILLISECONDS.sleep(500);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

Método auxiliar (evita repetir try/catch)

```
private static void pausar(int milissegundos) {
    try {
        Thread.sleep(milissegundos);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
// uso: pausar(500);
```

💡 `InterruptedException` é uma exceção **checada** — o compilador obriga o tratamento. Em código profissional, prefira `Thread.currentThread().interrupt()` a `e.printStackTrace()` (o IntelliJ inclusive sinaliza o `printStackTrace` com warning).

15. Datas em Java

A API moderna (desde o Java 8) fica no pacote `java.time` — **prefira sempre ela** à antiga `java.util.Date`.

LocalDate, LocalTime, LocalDateTime

```
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;

LocalDate hoje = LocalDate.now();           // 2026-07-05
LocalTime agora = LocalTime.now();          // 21:45:30.123
LocalDateTime dataHora = LocalDateTime.now(); // 2026-07-05T21:45:30

LocalDate natal = LocalDate.of(2026, 12, 25); // data específica
```

Extraindo partes

```
hoje.getDayOfMonth() // 5
hoje.getMonthValue() // 7
hoje.getYear()       // 2026
hoje.getDayOfWeek()  // SUNDAY
```

Operações com datas

```
hoje.plusDays(10)    // soma 10 dias
hoje.minusMonths(2)  // subtrai 2 meses
hoje.plusYears(1)    // soma 1 ano

// Diferença entre datas:
import java.time.temporal.ChronoUnit;
long dias = ChronoUnit.DAYS.between(hoje, natal);
```

Formatando para exibição (dd/MM/yyyy)

```
import java.time.format.DateTimeFormatter;

DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy");
System.out.println(hoje.format(fmt)); // 05/07/2026
```

Convertendo String para data

```
LocalDate data = LocalDate.parse("25/12/2026", fmt);
```

💡 **Insight de mercado:** a API antiga (`java.util.Date` , `Calendar`) ainda aparece muito em **sistemas legados bancários** — vale reconhecê-la, mas todo código novo deve usar `java.time` . Essa distinção costuma cair em entrevistas.

16. Dicas profissionais e insights

Código

1. **Valide toda entrada do usuário.** Loop `while` insistindo até receber dado válido é o padrão profissional — nunca confie que o usuário digitará certo.
2. **Curto-circuito é ferramenta de segurança**, não só otimização: `s.length() != 1 || s.charAt(0)...` — a primeira condição protege a segunda de exception.
3. **Nomeie pelo significado.** `produto` para multiplicação, `tabuada` se representa a tabuada — nunca `soma` guardando um produto. Nome errado é bug de comunicação.
4. **Some as partes para validar o todo:** em contadores por categoria (vogais + consoantes + outros), a soma deve bater com `length()` . Teste mental barato que pega bugs de lógica.
5. **Off-by-one:** sempre pergunte "esse loop roda quantas vezes?" — `<` vs `<=` muda o resultado.
6. **Flags precisam de reset.** Em loops de tentativa (validação de senha, menus), reinicialize as flags booleanas a cada iteração.
7. **Divisão inteira:** `int / int` trunca. Force `double` com cast: `(double) soma / cont` .
8. `.equals()` para Strings, `==` para primitivos. Decorar isso evita horas de debug.

Ferramentas e fluxo

1. `Ctrl + Alt + L` no IntelliJ reformata o código no padrão Java — use antes de todo commit.
2. **Commits pequenos e frequentes** com mensagens descritivas ("Adiciona Ex10 - operações escaladas") valem mais no portfólio que um commit gigante semanal.
3. `git pull` antes de começar a sessão evita rejeição de push quando algo foi alterado pelo site do GitHub.
4. **Nomeie arquivos com zero à esquerda** (`Ex01` , `Ex02` ... `Ex10`) para a ordenação alfabética coincidir com a numérica.

Carreira (mercado corporativo/bancário)

1. **Leia stack traces de baixo para cima na sua parte:** a primeira linha diz o que aconteceu (`SecurityException: Prohibited package name`), as linhas com suas classes dizem onde.
2. **Warnings do IntelliJ são mini-aulas gratuitas.** O sublinhado amarelo em `printStackTrace()` ensina que log profissional usa frameworks (SLF4J/Log4j) — investigue cada warning ao menos uma vez.
3. **Repositório organizado é portfólio.** README claro, pacotes coerentes, uma classe por exercício e commits descritivos mostram profissionalismo antes mesmo de abrirem seu código.
4. **Sistemas bancários são Java-heavy e legados.** Conhecer tanto a forma moderna (`java.time` , streams) quanto reconhecer a antiga (`Date` , loops clássicos) é diferencial real em consultoria.

📖 Referências de estudo

- Curso Java COMPLETO — Nélío Alves (DevSuperior)
- Documentação oficial do Java (Oracle)
- Java Tutorials — Oracle

Apostila em constante evolução — atualizada conforme o avanço nos estudos.

Jader Silva · [GitHub](#) · [LinkedIn](#)

